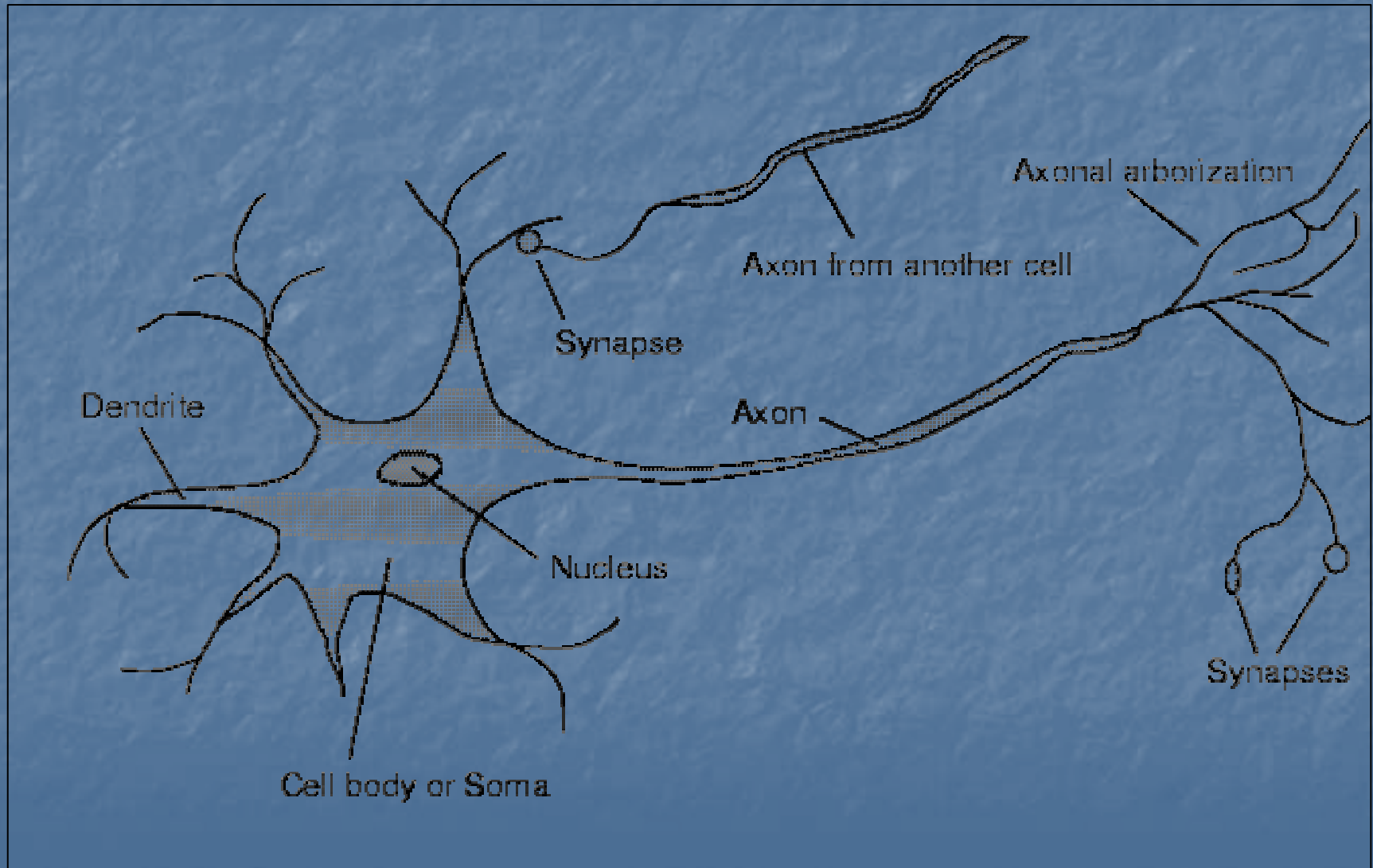


Neuron Model and Network Architectures

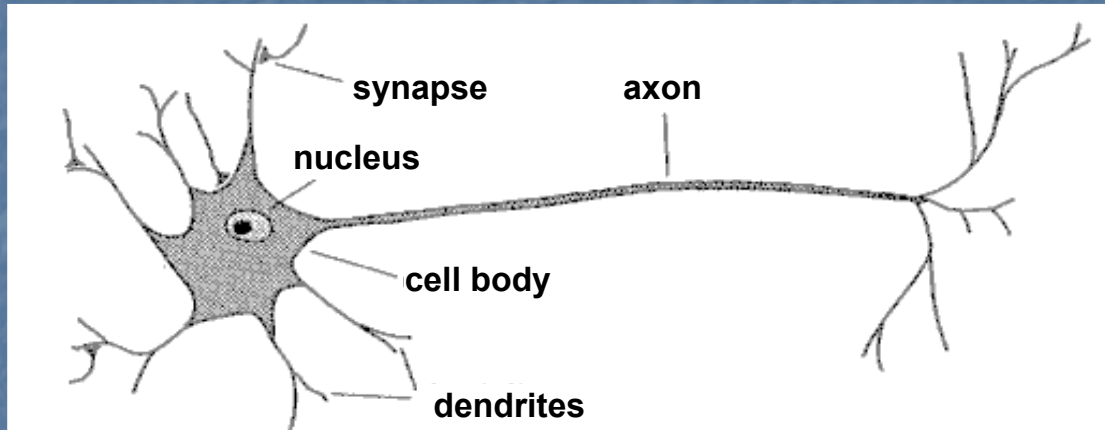
Biological inspirations

- Some numbers:
 - The human brain contains about 10 billion nerve cells (neurons)
 - Each neuron is connected to the others through 10000 synapses
- Properties of the brain:
 - It can learn, reorganize itself from experience
 - It adapts to the environment
 - It is robust and fault tolerant

Human Nerve Cell



Biological neuron



- A neuron has
 - A branching input (dendrites)
 - A branching output (the axon)
- The information circulates from the dendrites to the axon via the cell body
 - Axon connects to dendrites via synapses
 - Synapses vary in strength
 - Synapses may be excitatory or inhibitory

Connectionist Models

Consider humans

- Neuron switching time $\sim .001$ second
- Number of neurons $\sim 10^{10}$
- Connections per neuron $\sim 10^{4-5}$
- Scene recognition time $\sim .1$ second
- 100 inference step does not seem like enough

must use lots of parallel computation!

Properties of artificial neural nets (ANNs)

- Many neuron-like threshold switching units
- Many weighted interconnections among units
- Highly parallel, distributed process
- Emphasis on tuning weights automatically

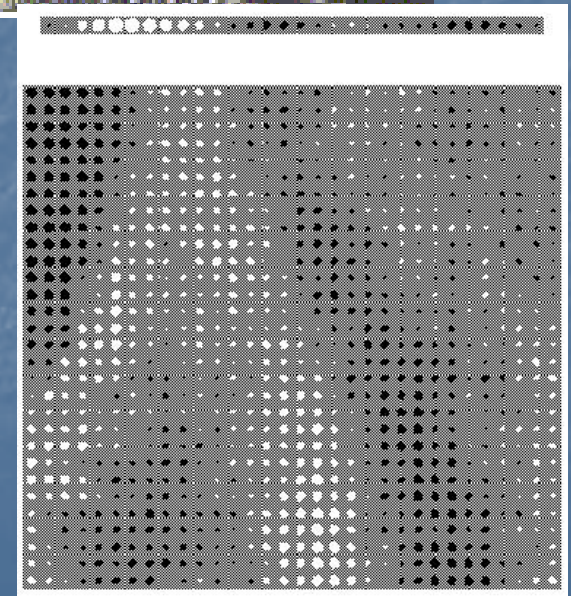
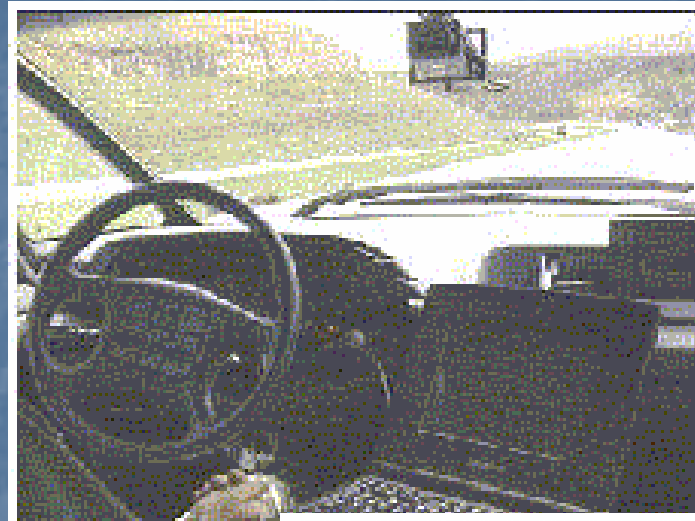
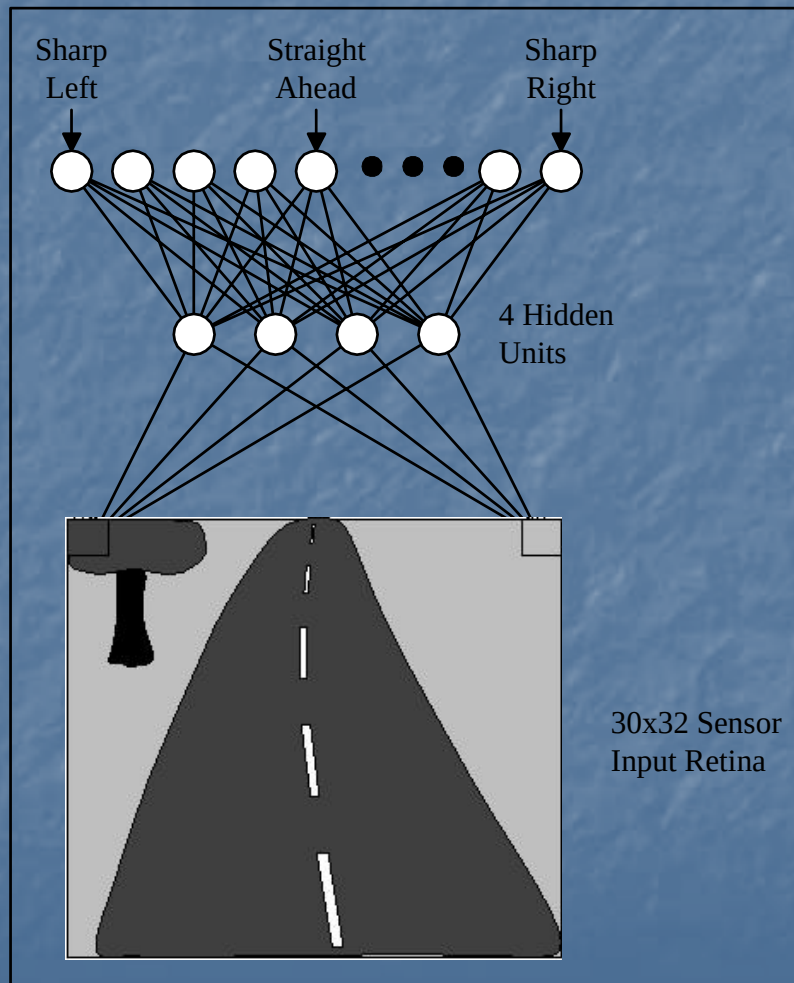
When to Consider Neural Networks

- Input is high-dimensional discrete or real-valued (e.g., raw sensor input)
- Output is discrete or real valued
- Output is a vector of values
- Possibly noisy data
- Form of target function is unknown
- Human readability of result is *unimportant*

Examples

- Speech phoneme recognition
- Image classification
- Financial prediction

ALVINN drives 70 mph on highways



Neural Networks

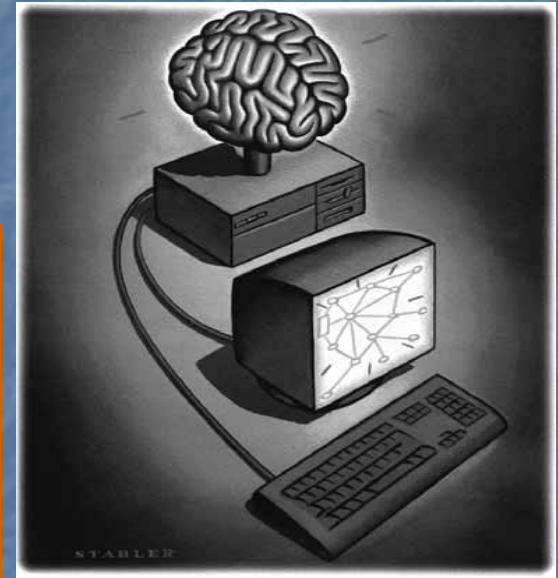
What is a Neural Network?

- Biologically motivated approach to machine learning

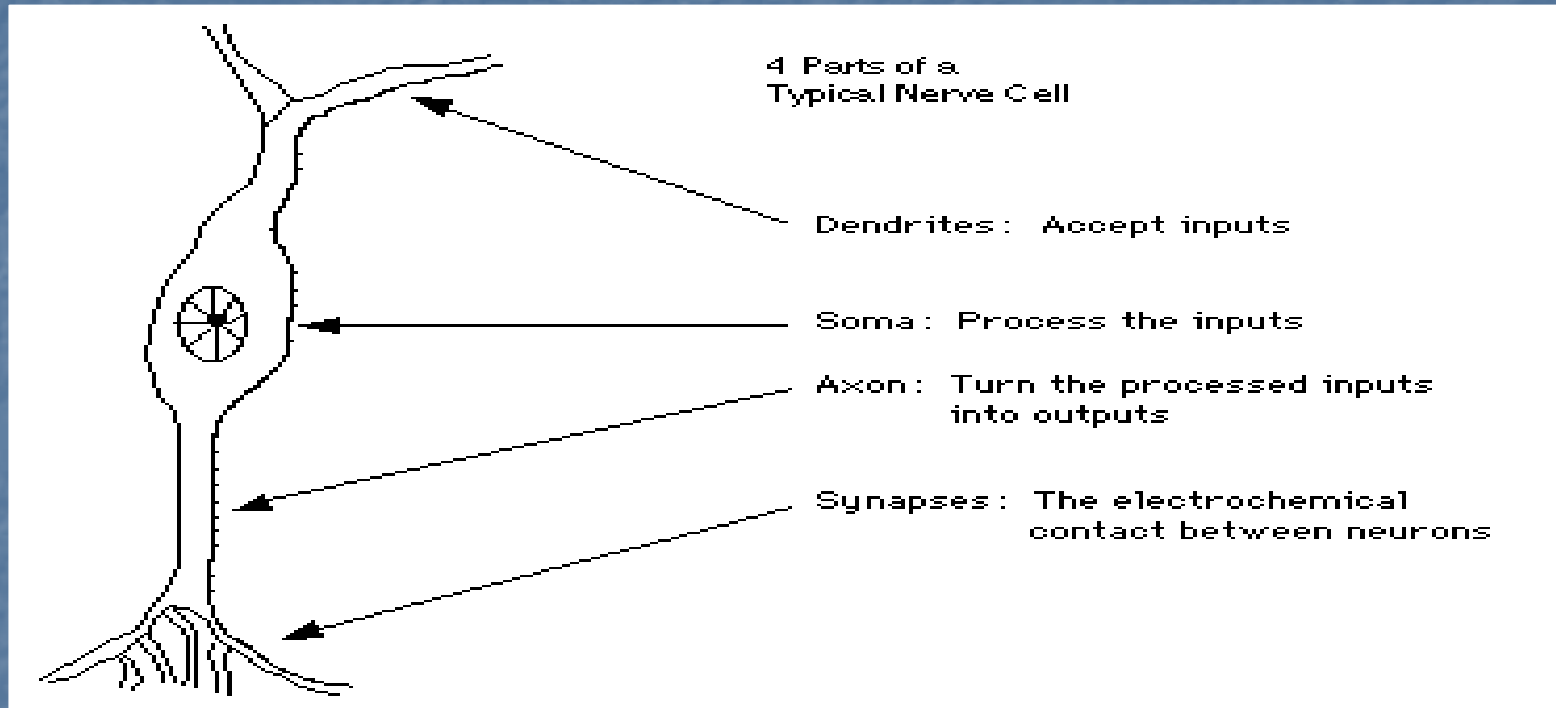
Similarity with biological network

Fundamental processing elements of a neural network is a neuron:

1. Receives inputs from other source
2. Combines them in someway
3. Performs a generally nonlinear operation on the result
4. Outputs the final result

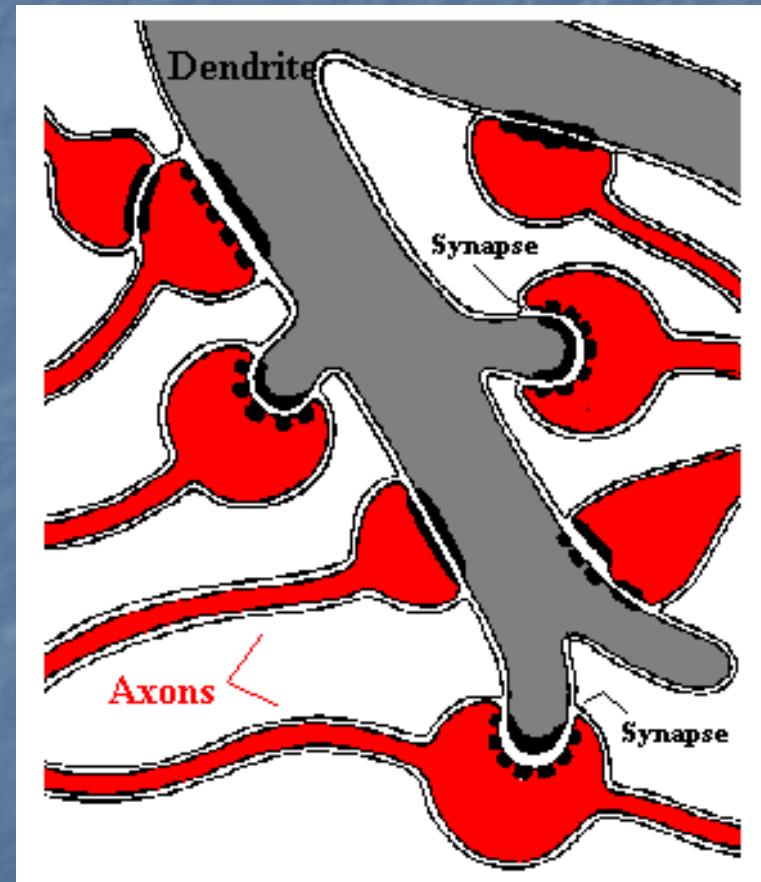
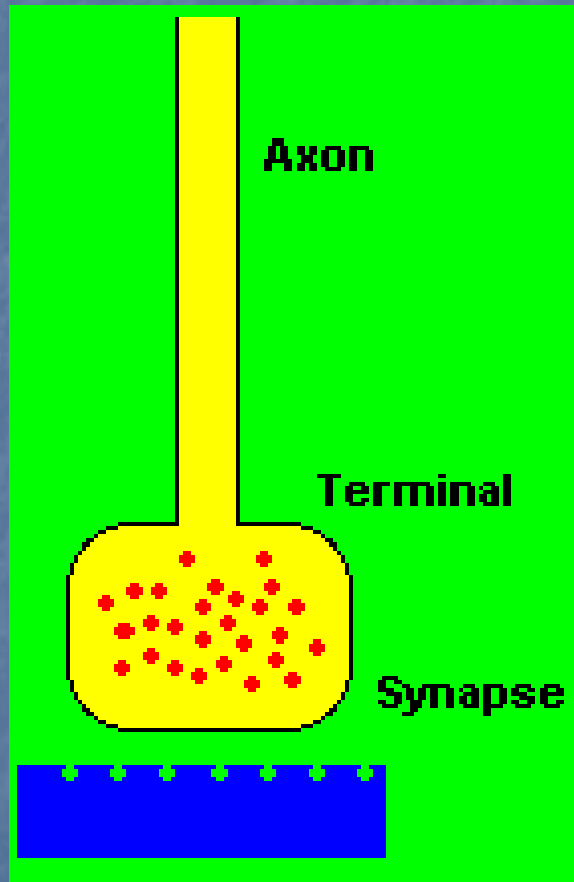


Similarity with Biological Network



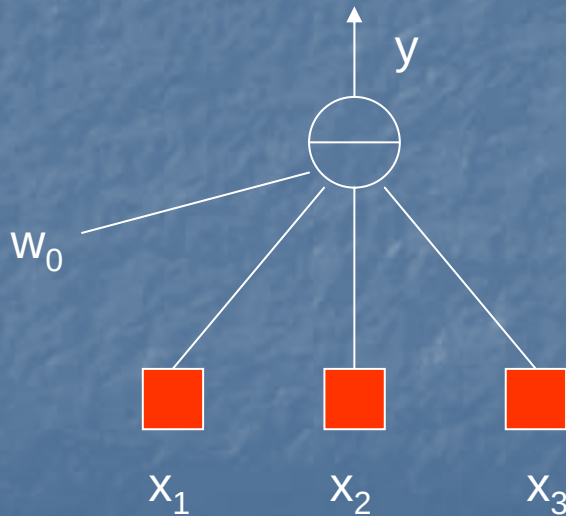
- Fundamental processing element of a neural network is a neuron
- A human brain has 100 billion neurons
 - An ant brain has 250,000 neurons

Synapses, the basis of learning and memory



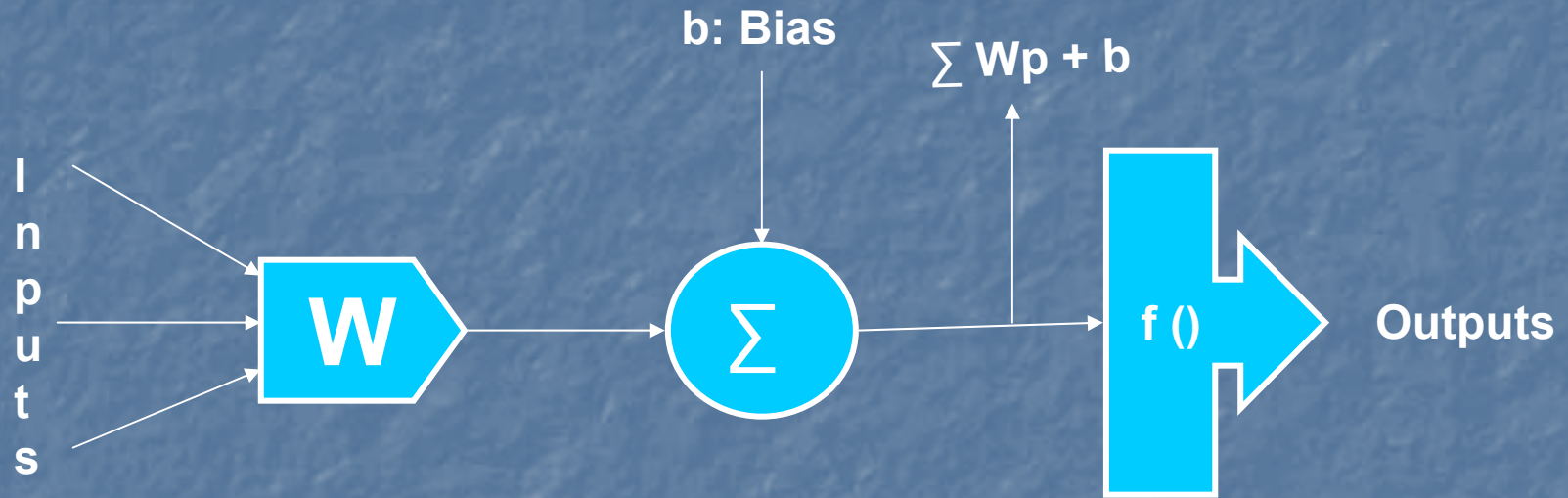
What is an artificial neuron ?

- Definition: Nonlinear, parameterized function with restricted output range



$$y = f \left(w_0 + \sum_{i=1}^{n-1} w_i x_i \right)$$

Basic Neural Network



Inputs – normally a vector of measured parameters

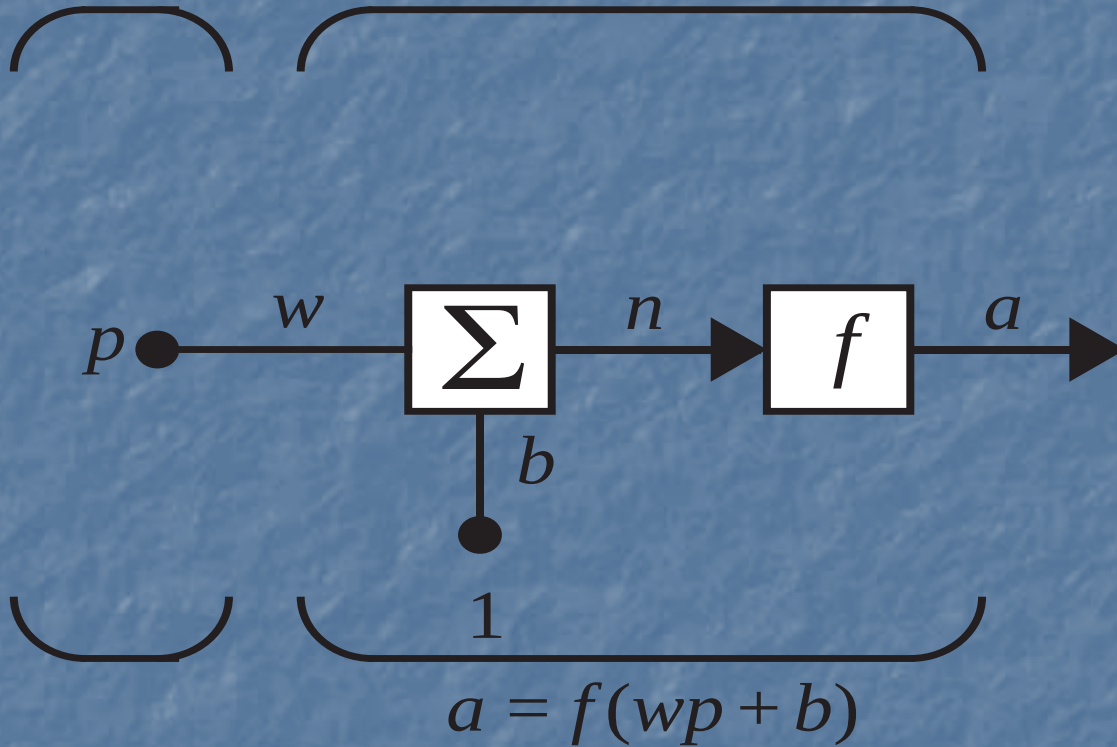
Bias – may / may not be added

f() – transfer or activation function

$$\text{Outputs} = \mathbf{f} \left(\sum \mathbf{W}^T \mathbf{p} + \mathbf{b} \right)$$

Inputs

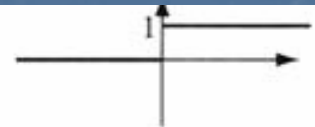
General Neuron



Activation Functions

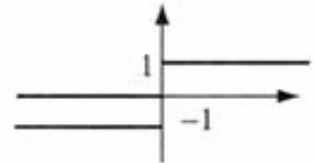
Hard Limit

$$y = \begin{cases} 1 & \text{if } net \geq 0 \\ 0 & \text{if } net < 0 \end{cases}$$



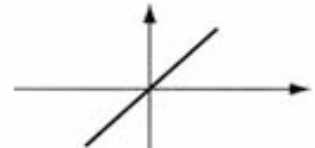
Symmetrical Hard Limit

$$y = \begin{cases} 1 & \text{if } net \geq 0 \\ -1 & \text{if } net < 0 \end{cases}$$



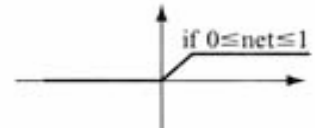
Linear

$$Y = net$$



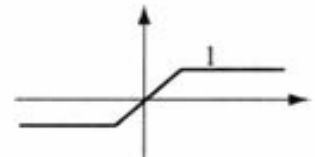
Saturating Linear

$$y = \begin{cases} 1 & \text{if } net > 1 \\ net & \text{if } 0 \leq net \leq 1 \\ 0 & \text{if } net < 0 \end{cases}$$



Symmetric Saturating Linear

$$y = \begin{cases} 1 & \text{if } net > 1 \\ net & \text{if } -1 \leq net \leq 1 \\ -1 & \text{if } net < -1 \end{cases}$$



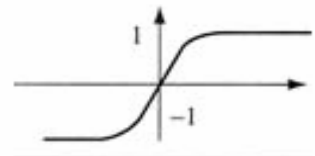
Log-Sigmoid

$$y = 1 / (1 + e^{-net})$$

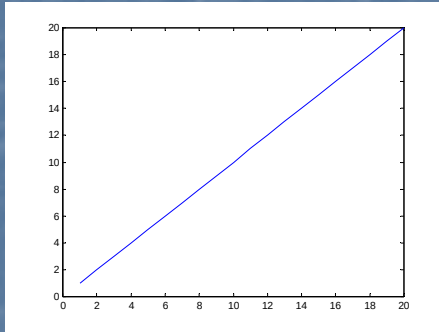


Hyperbolic Tangent Sigmoid

$$y = (e^{net} - e^{-net}) / (e^{net} + e^{-net})$$

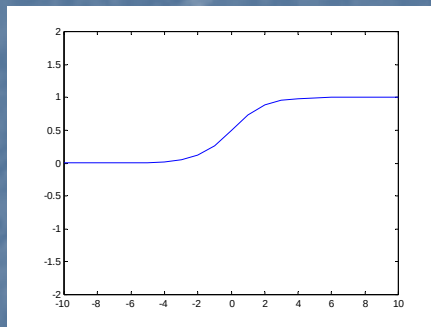


Activation functions



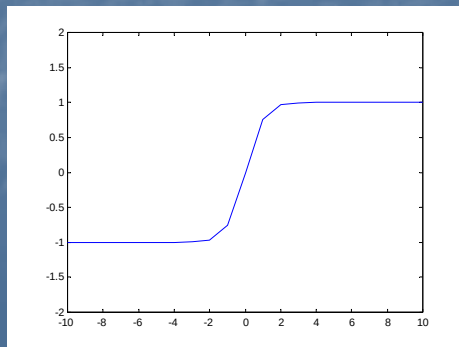
Linear

$$y = x$$



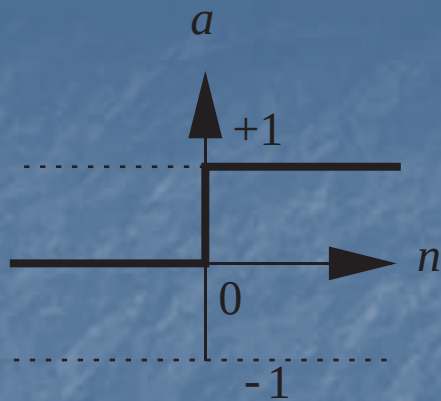
Logistic

$$y = \frac{1}{1 + \exp(-x)}$$



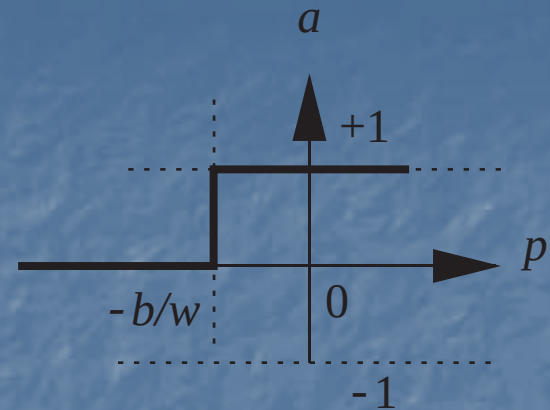
Hyperbolic tangent

$$y = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$$



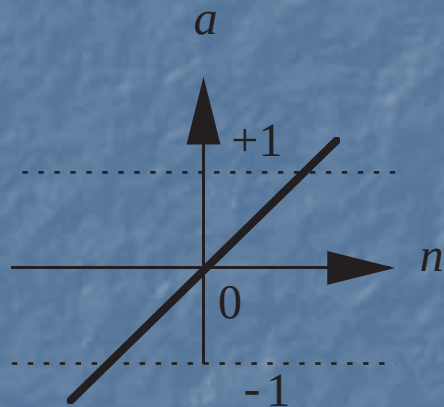
$$a = \text{hardlim}(n)$$

Hard Limit Transfer Function



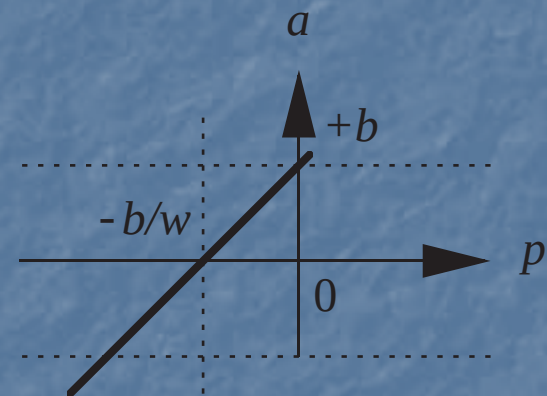
$$a = \text{hardlim}(wp + b)$$

Single-Input *hardlim* Neuron



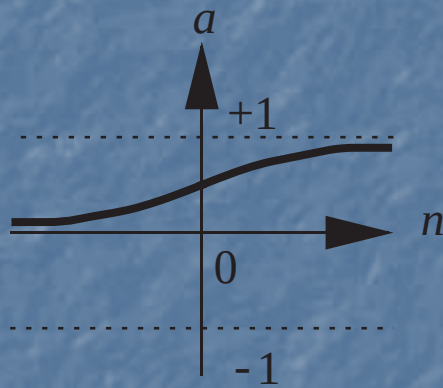
$$a = \text{purelin}(n)$$

Linear Transfer Function



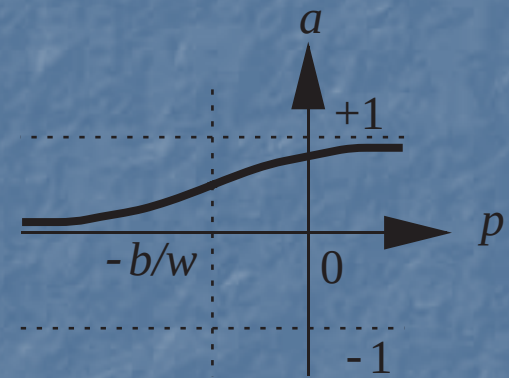
$$a = \text{purelin}(wp + b)$$

Single-Input *purelin* Neuron



$$a = \text{logsig}(n)$$

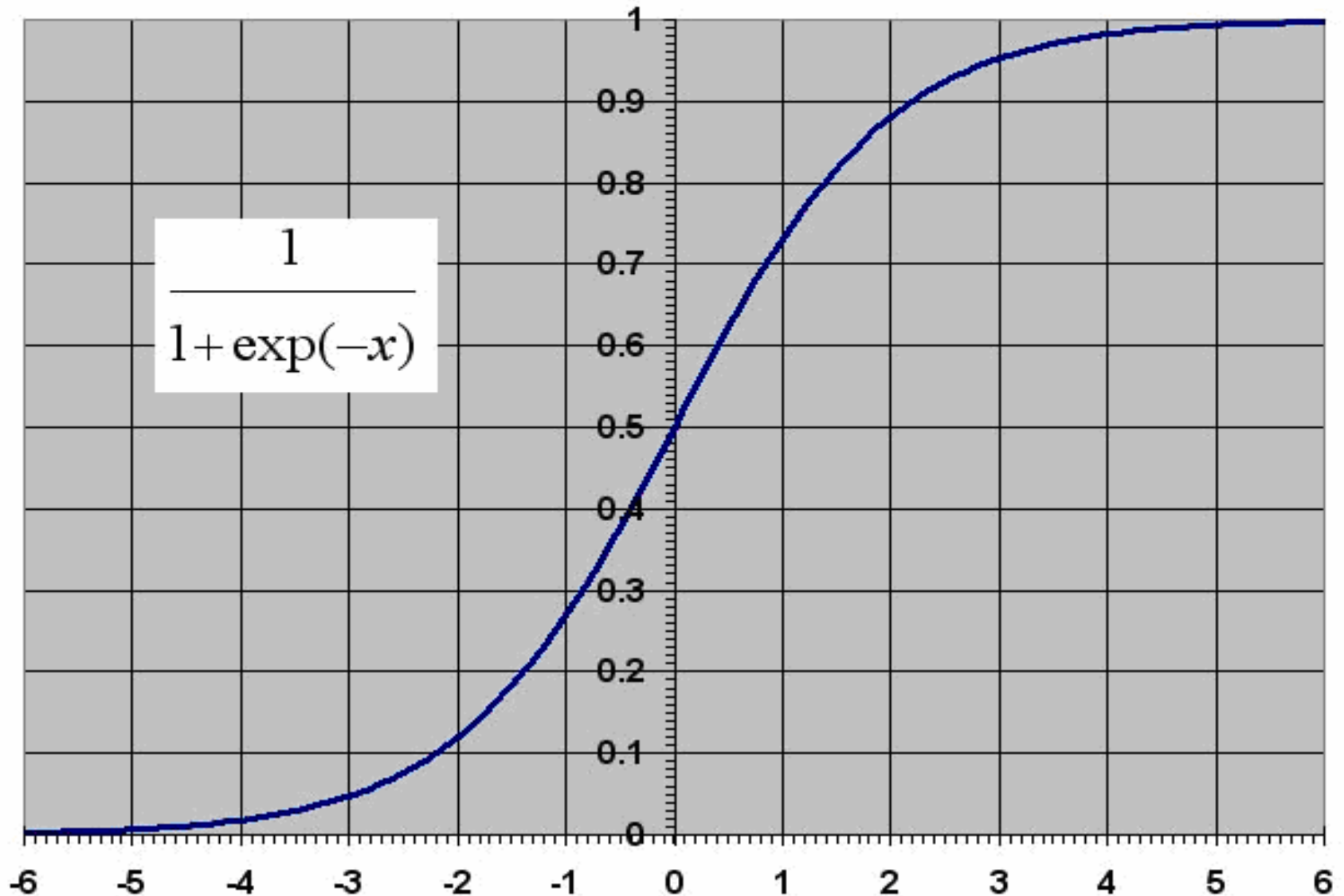
Log-Sigmoid Transfer Function



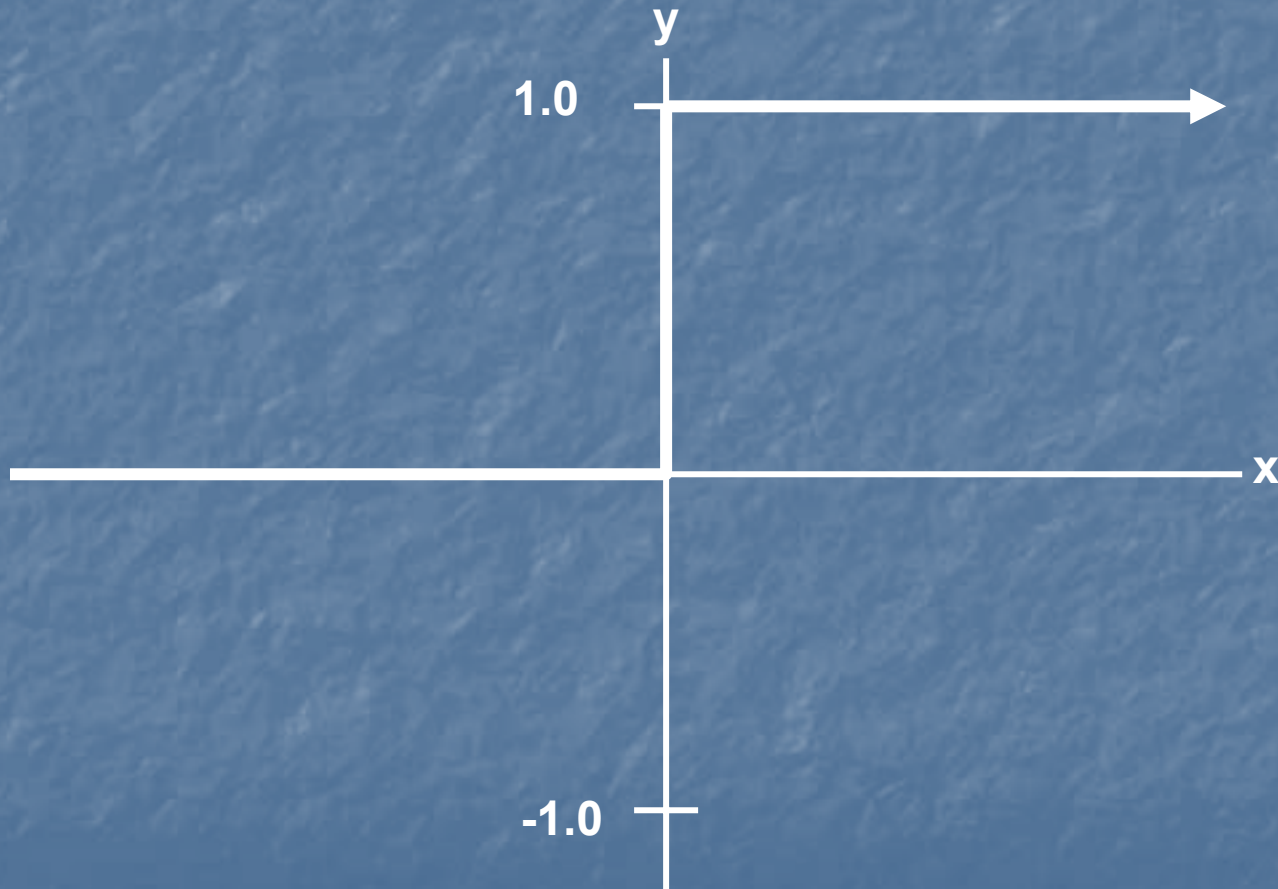
$$a = \text{logsig}(wp + b)$$

Single-Input *logsig* Neuron

Log Sigmoidal Function



Hard Limit Function



Neural Networks

- A mathematical model to solve engineering problems
 - Group of highly connected neurons to realize compositions of non linear functions
- Tasks
 - Classification
 - Discrimination
 - Estimation
- 2 types of networks
 - Feed forward Neural Networks
 - Recurrent Neural Networks

Learning

- The procedure that consists in estimating the parameters of neurons so that the whole network can perform a specific task
- Two general types of learning
 - The supervised learning
 - The unsupervised learning
- The Learning process (supervised):
 - Present the network a number of inputs and their corresponding outputs
 - See how closely the actual outputs match the desired ones
 - Modify the parameters to better approximate the desired outputs

Supervised learning

- The desired response of the neural network in function of particular inputs is well known.
- A “Professor” may provide examples and teach the neural network how to fulfill a certain task

Unsupervised learning

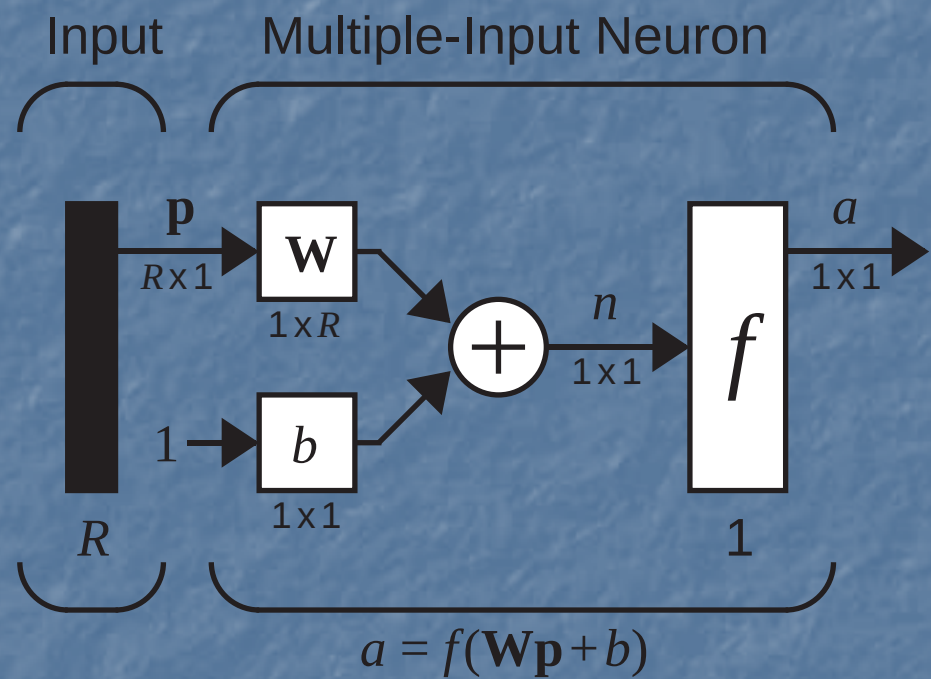
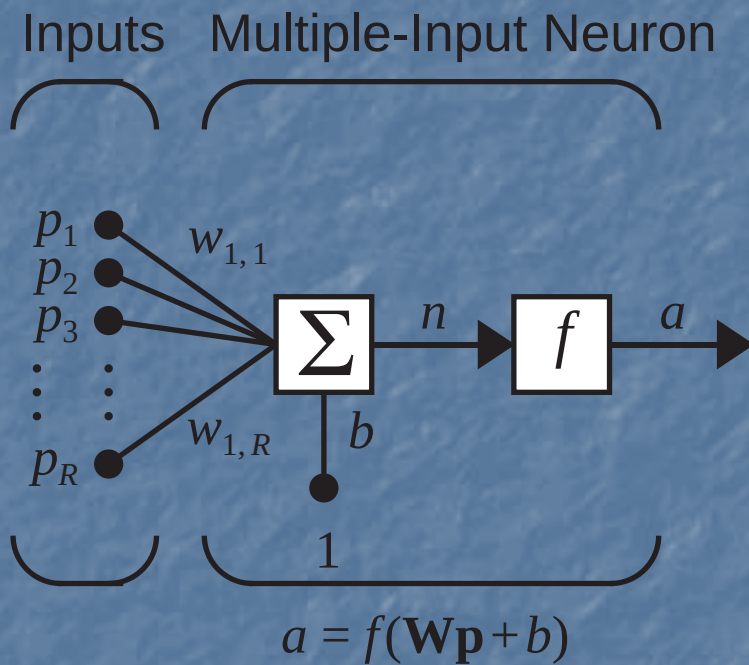
- Idea: group typical input data in function of resemblance criteria un-known a priori
- Data clustering
- No need of a professor:
 - The network finds itself the correlations between the data
 - Examples of such networks:
 - Kohonen feature maps

What do we need to use NN ?

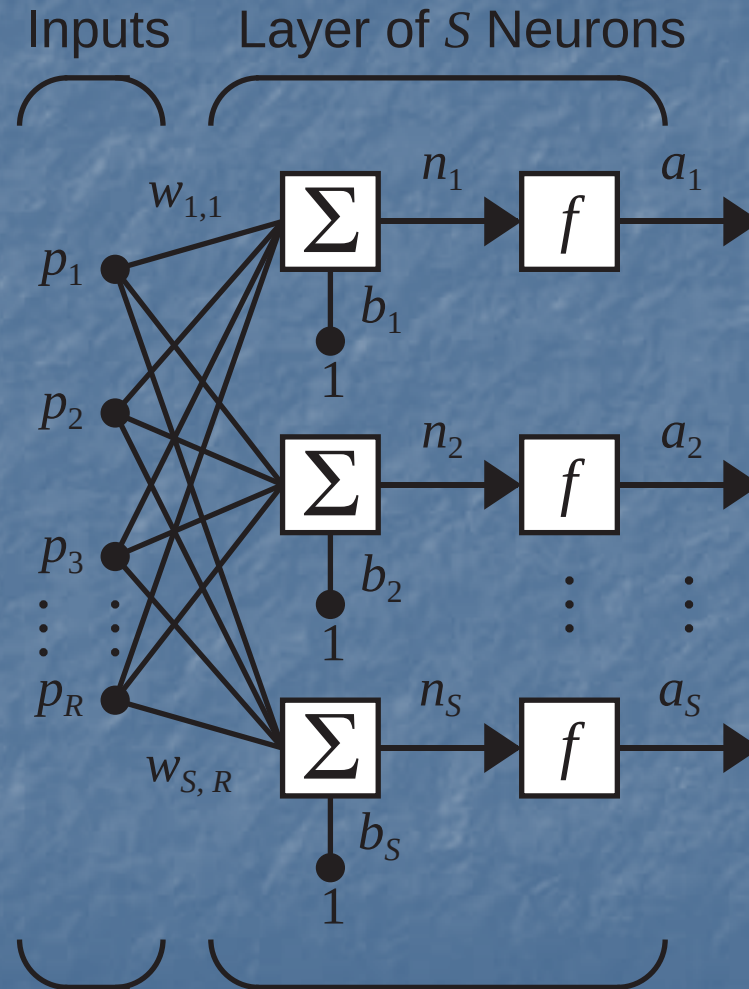
- (1) Determination of pertinent inputs
- (2) Collection of data for the learning and testing phase of the neural network
- (3) Finding the optimum number of hidden nodes
- (4) Estimate the parameters (Learning)
- (5) Evaluate the performances of the network
- (6) IF performances are not satisfactory then review all the precedent points

Classical neural architectures

- Perceptron
- Multi-Layer Perceptron
- Radial Basis Function (RBF)
- Kohonen Features maps
- Other architectures
 - An example: Shared weights neural networks

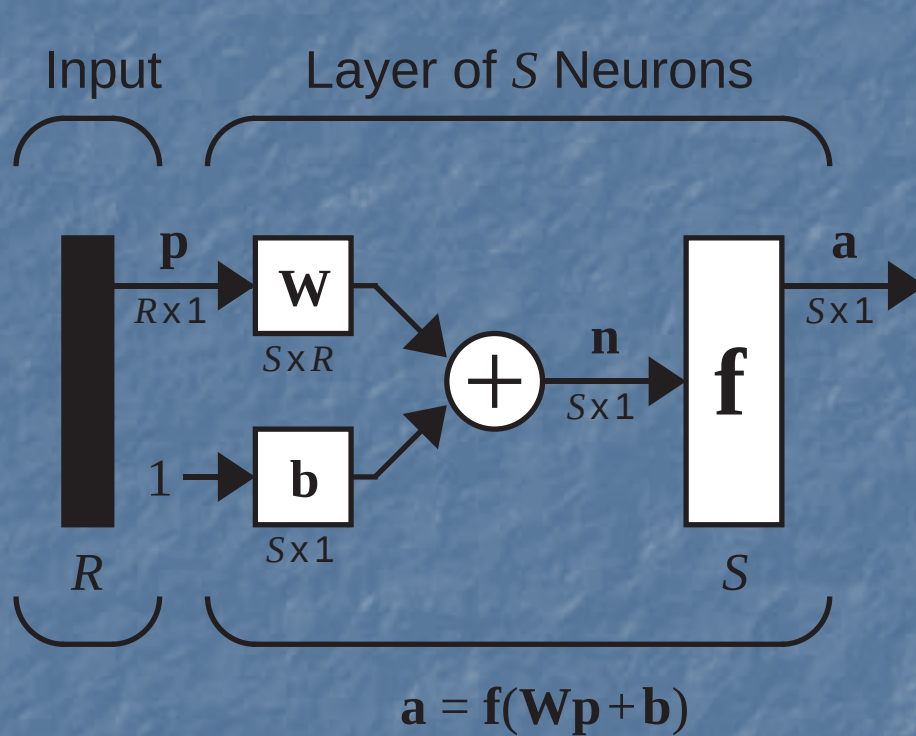


Layer of Neurons



$$\mathbf{a} = \mathbf{f}(\mathbf{W}\mathbf{p} + \mathbf{b})$$

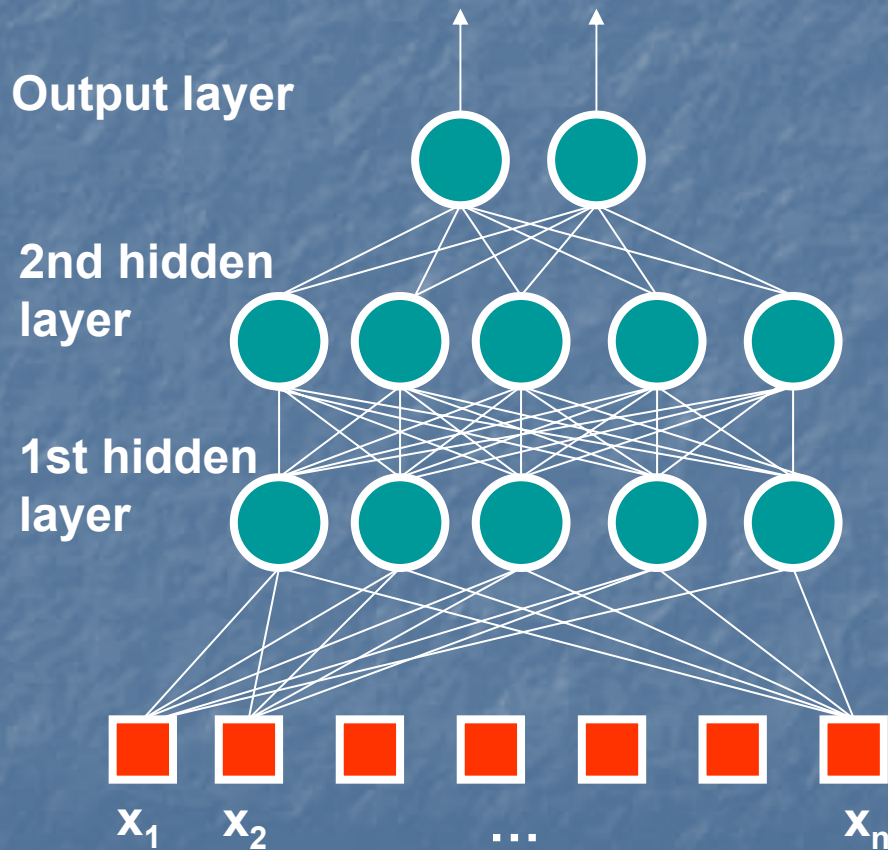
Abbreviated Notation



$$\mathbf{W} = \begin{bmatrix} w_{1,1} & w_{1,2} & \cdots & w_{1,R} \\ w_{2,1} & w_{2,2} & \cdots & w_{2,R} \\ \vdots & \vdots & & \vdots \\ w_{S,1} & w_{S,2} & \cdots & w_{S,R} \end{bmatrix}$$

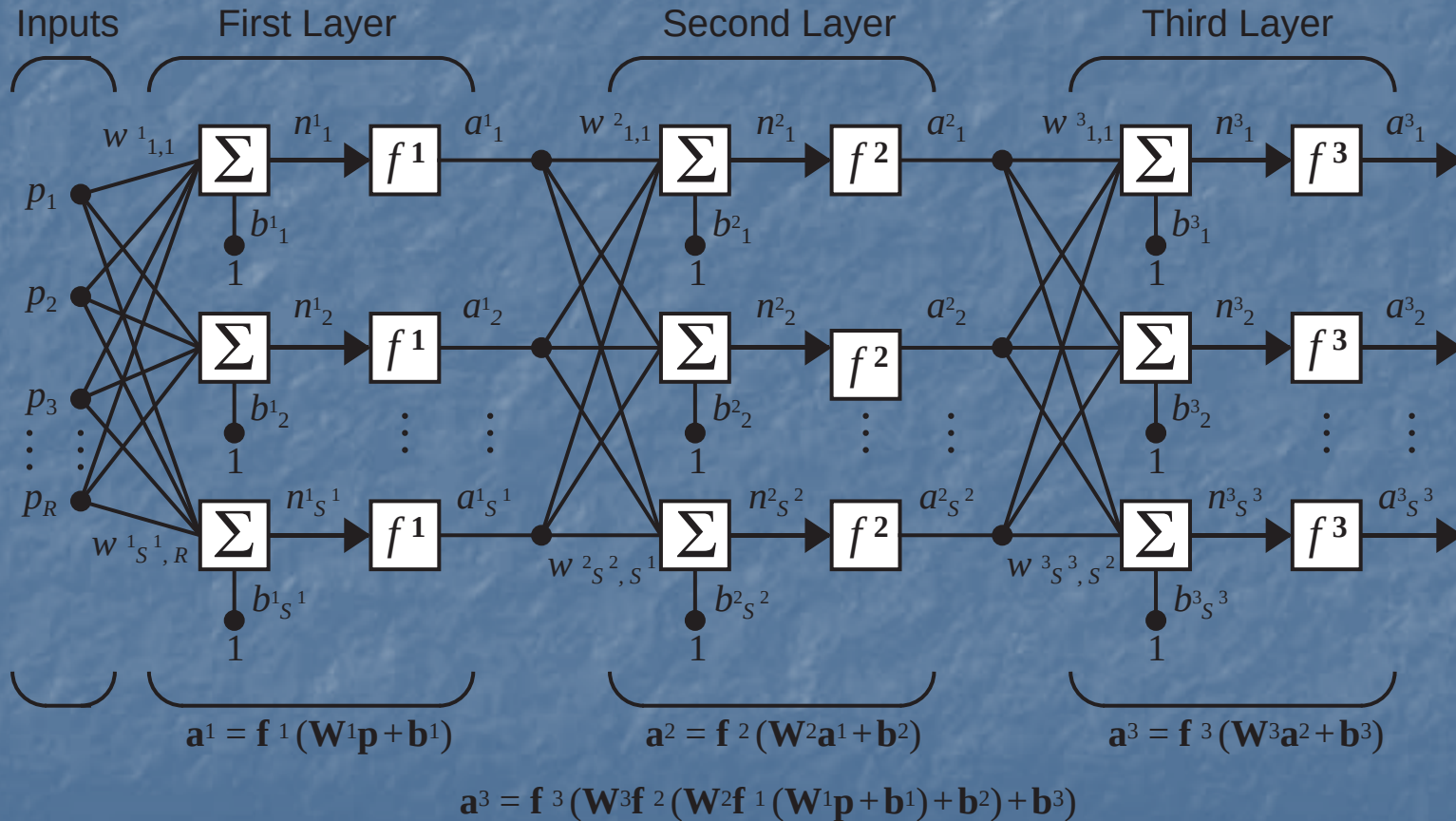
$$\mathbf{p} = \begin{bmatrix} p_1 \\ p_2 \\ \vdots \\ p_R \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_S \end{bmatrix} \quad \mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_S \end{bmatrix}$$

Feedforward Neural Networks

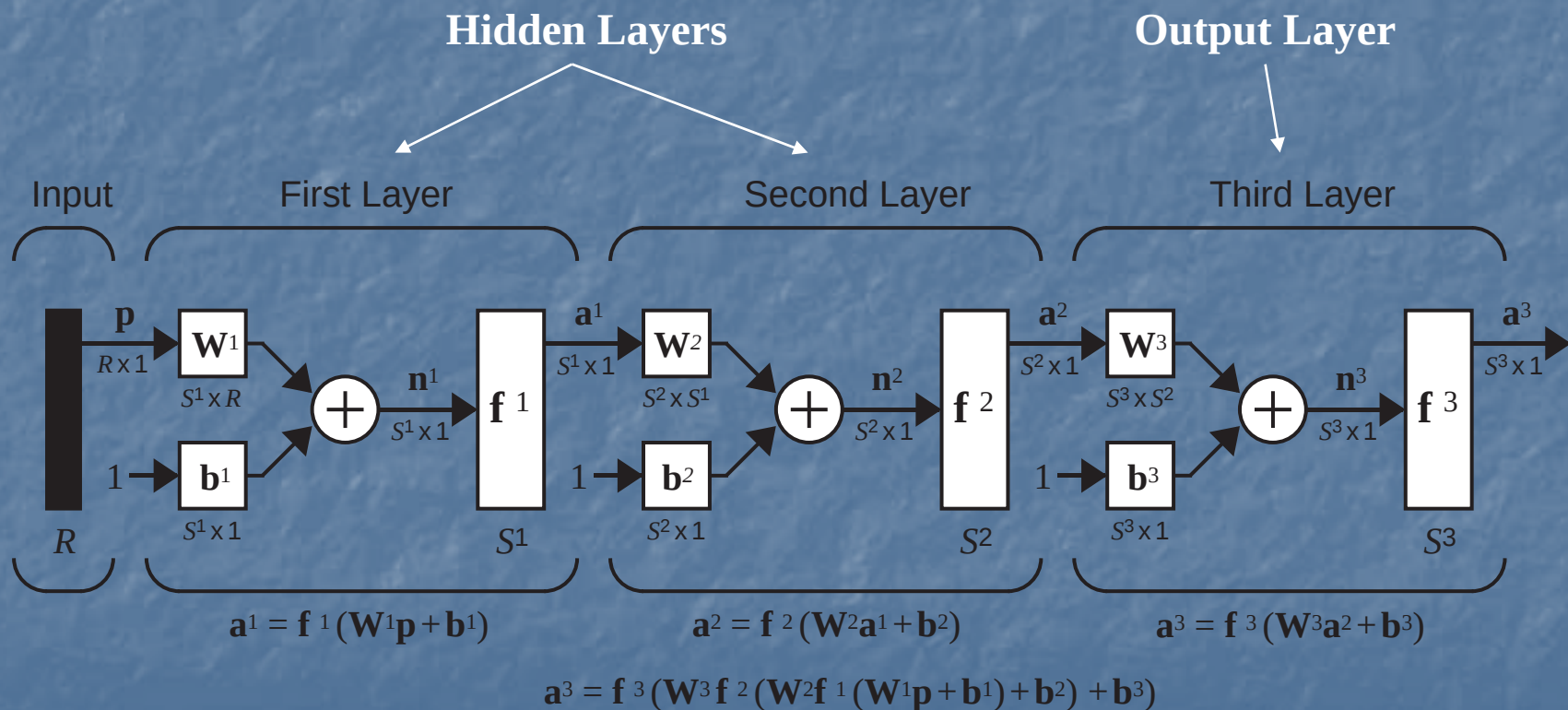


- The information is propagated from the inputs to the outputs
- Computations of **No** non linear functions from **n** input variables by compositions of **Nc** algebraic functions
- Time has no role (NO cycle between outputs and inputs)

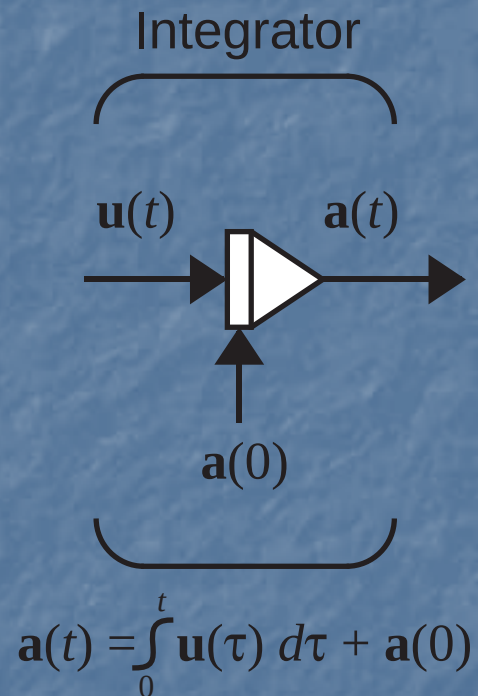
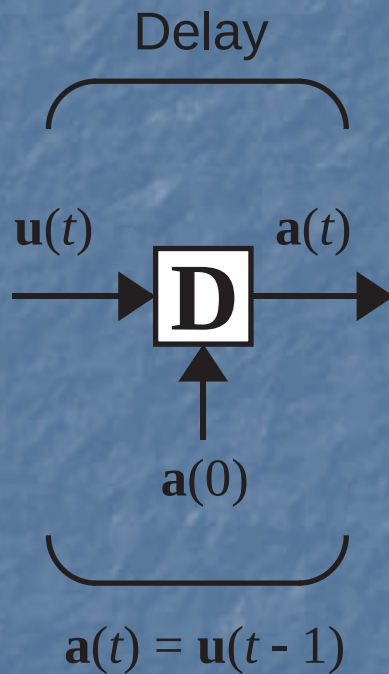
Multilayer Network



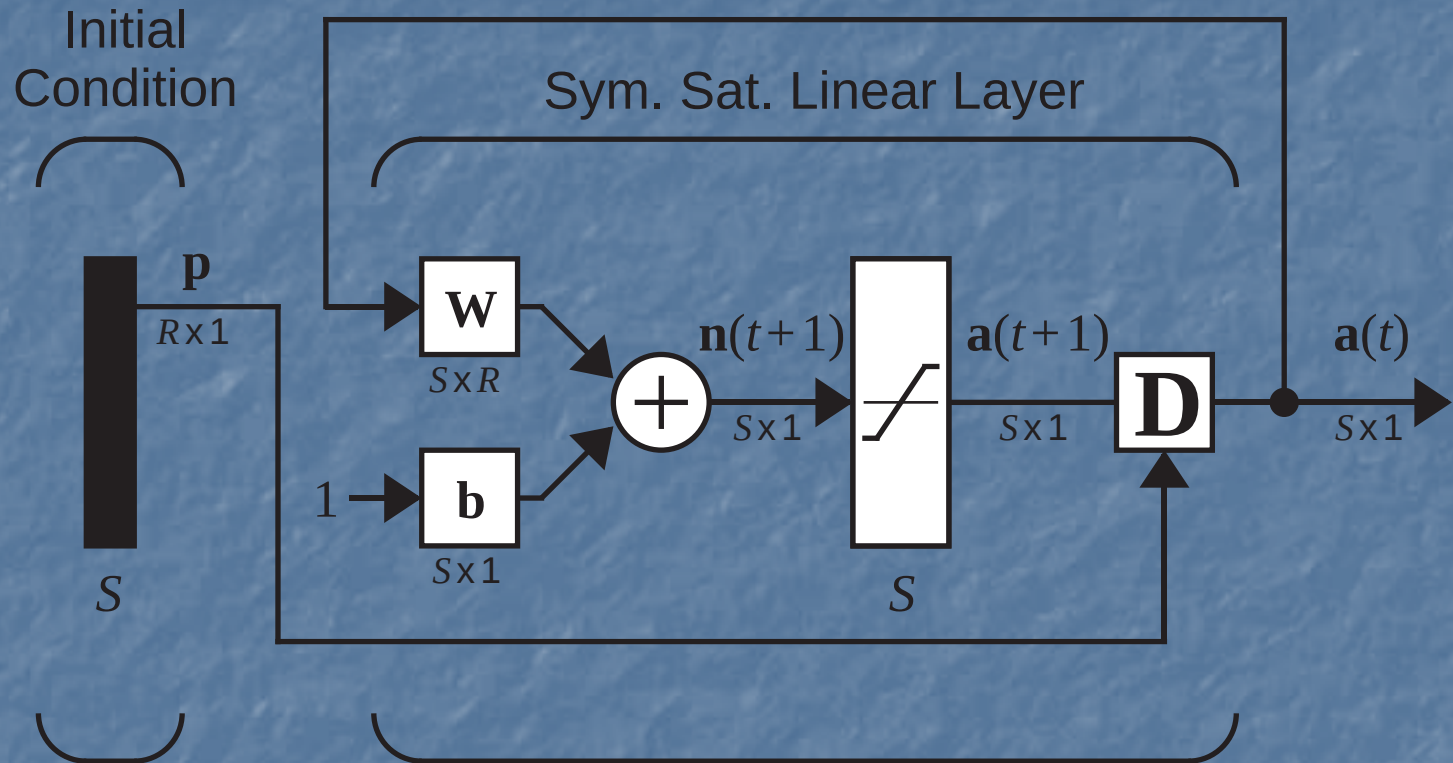
Abbreviated Notation



Delays and Integrators



Recurrent Network



$$a(0) = p \quad a(t+1) = \text{satlin}(Wa(t) + b)$$

$$a(1) = \text{satlins}(W a(0) + b) = \text{satlins}(W p + b)$$

$$a(2) = \text{satlins}(W a(1) + b)$$

Brief History of ANN

- **McCulloch and Pitts (1943)** designed the first neural network.
- **Hebb (1949)** who developed the first learning rule. If two neurons were active at the same time then the strength between them should be increased.
- **Rosenblatt (1958)** – introduced the concept of a perceptron which performed pattern recognition.
- **Widrow and Hoff (1960)** introduced the concept of the ADALINE (ADaptive Linear Element). The training rule was based on the idea of Least-Mean-Squares learning rule which minimizing the error between the computed output and the desired output.
- **Minsky and Papert (1969)** stated that the perceptron was limited in its ability to recognize features that were separated by linear boundaries. “Neural Net Winter”
- **Kohonen and Anderson** – independently developed neural networks that acted like memories.
- **Webros(1974)** – developed the concept of back propagation of an error to train the weights of the neural network.
- **McCelland and Rumelhart (1986)** published the paper on back propagation algorithm. “Rebirth of neural networks”.
- **Today** - they are everywhere a decision can be made.